为什么会跨域

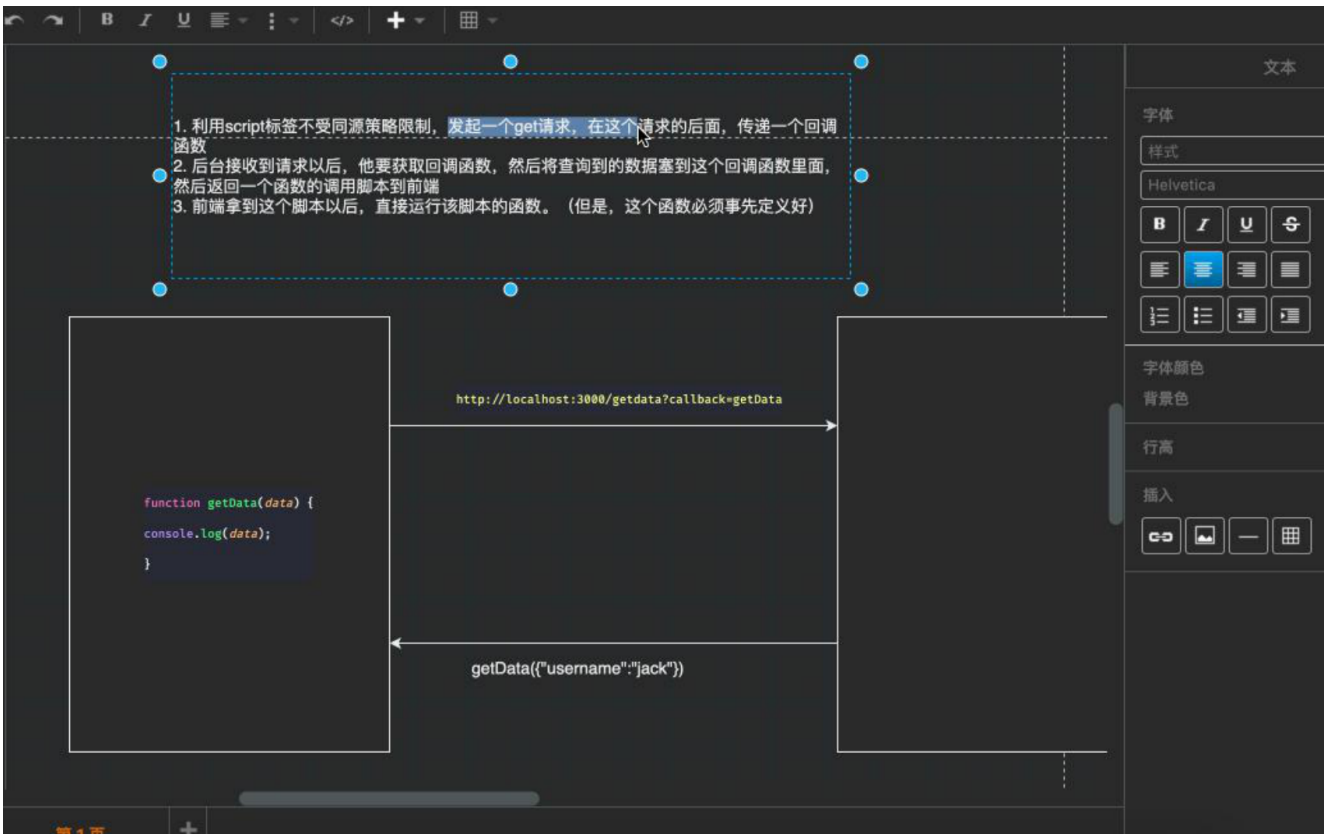
因为浏览器同源策略的限制 域名协议端口号不一样的时候就存在跨域问题

当请求 看到CORS error的时候，就说明已经跨域了 Cross Origin Resource Share

常见的解决跨域的方式 CORS (后台) 代理服务器 jsonp

核心

利用script标签不受同源策略的限制，发起一个GET请求，在这个请求的后面，传递一个回调函数2.当后台收到这个请求后，要获取回调函数，然后将查询到的数据塞到这个回调函数里面，然后返回一个函数调用脚本到前端3.前端拿到这个脚本以后，直接运行该脚本的函数（但是这个脚本必须事先定义好）



封装自己的jsonp函数

```
<script>
        const jsonp = (url) => {
            //创建一个script标签
            let fn = 'itcast' + Math.random().toString().replace('.','')
            let script = document.createElement('script')
            let myurl = url + '?callback=' + fn
            console.log(fn);
```

```
            script.src = myurl
            window[fn] = function (data) { console.log(data); }
            document.body.append(script)
            //删除节点
            script.onload = function () {
                script.remove()
                //删除对象属性
                delete window[fn]
            }
        }
        jsonp('http://192.168.14.86:3000/getdata')
        console.log(window);

    </script>
```

后端cors解决跨域问题

```
npm i cors --save // 下载 cors 第三方模块
const cors=require('cors')//引入模块
app.use(cors()) //使用中间件

```

node原生的方式解决跨域问题

```
app.all('*', function(req, res, next) {
  res.header("Access-Control-Allow-Origin", "*");
  res.header("Access-Control-Allow-Headers", "Content-Type,Content-Length, Authorization, Accept,X-Requested-With");
  res.header("Access-Control-Allow-Methods","PUT,POST,GET,DELETE,OPTIONS");
  if(req.method=="OPTIONS") res.send(200);/*让options请求快速返回*/
  else next();
});
```

webpack设置代理的方式解决跨域

因为服务器和服务器之间不存在跨域，跨域只存在在浏览器中利用这一点可以在webpack中设置

官网有详细的解说，在vue中是vueconfig.js

```
proxy: {
    '/api': {
      // 代理的目标地址
      target: 'http://59.110.172.233:3000',
      // http://ihrm-java.itheima.net',
      // 是否允许跨域
      changeOrigin: true
    }
  }
```